

Início

Limpeza

Modelo

Outros

SPACESHIP TITANIC

RESULTADOS

Acurácia: 0.7453

Relatório:

	precision	recall	f1-score	support
False	0.74	0.75	0.74	863
True	0.75	0.74	0.75	876
accuracy			0.75	1739
macro avg	0.75	0.75	0.75	1739
weighted avg	0.75	0.75	0.75	1739

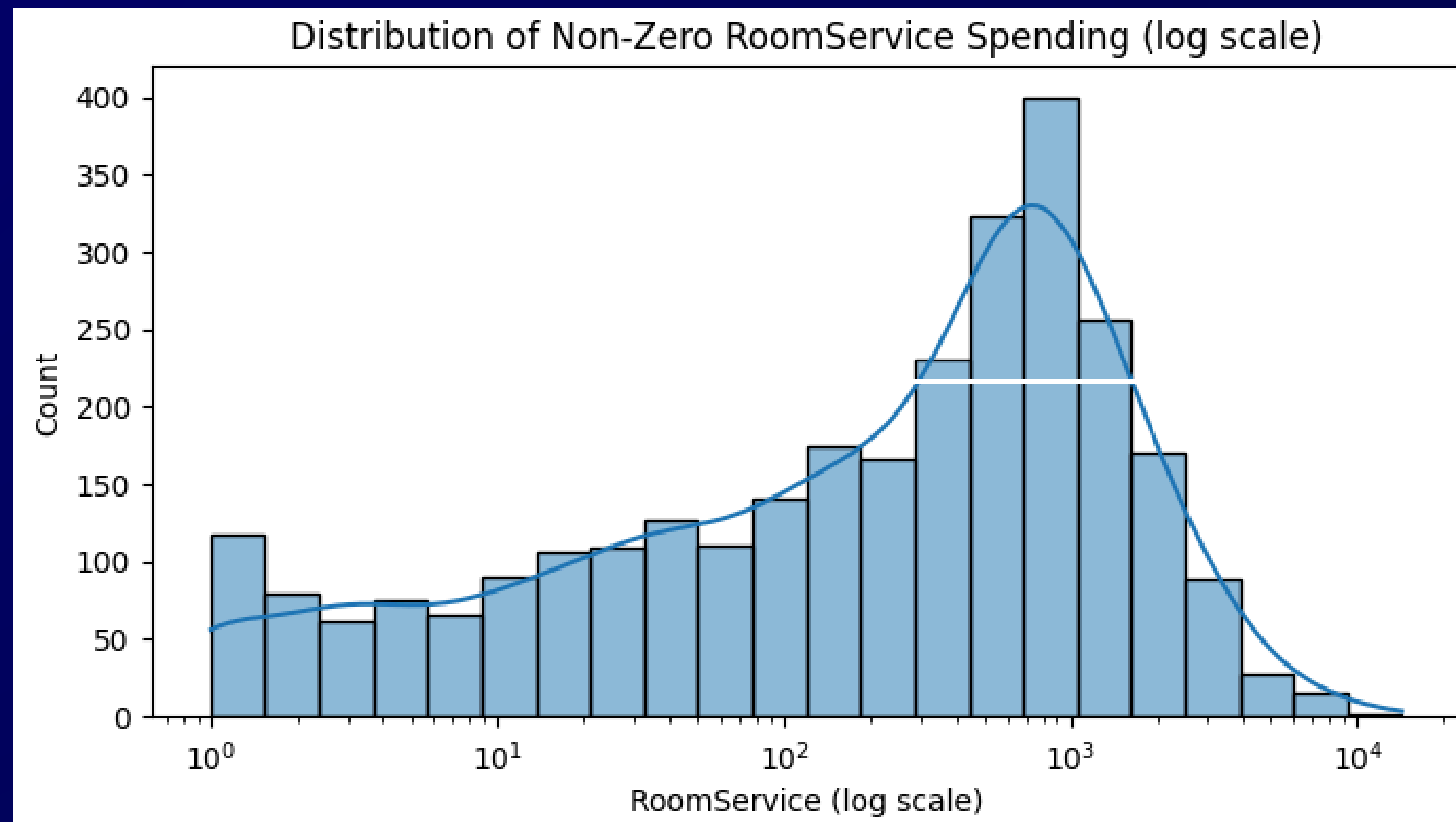
ACURÁCIA 0.745

LIMPEZA DE DADOS

	Variável	Descrição	Tipo	Mensuração	Valores possíveis
0	PassengerId	Identificador único de cada passageiro a bordo (formato: grupo_pessoa)	object	Categórica	0001_01, 0002_01, 0003_01, 0003_02, 0004_01, 0005_01, 0006_01, 0006_02, 0007_01, 0008_01
1	HomePlanet	Planeta de origem do passageiro (ex: Europa, Earth, Mars)	object	Categórica	Europa, Earth, Mars, nan
2	CryoSleep	Indica se o passageiro estava em sono criogênico durante a viagem (True = sim, False = não)	object	Categórica	False, True, nan
3	Cabin	Identificação da cabine do passageiro (contém o número, lado e deck — ex: B/0/P)	object	Categórica	B/0/P, F/0/S, A/0/S, F/1/S, F/0/P, F/2/S, G/0/S, F/3/S, B/1/P, F/1/P
4	Destination	Destino final da viagem interestelar (ex: TRAPPIST-1e, PSO J318.5-22, 55 Cancri e)	object	Categórica	TRAPPIST-1e, PSO J318.5-22, 55 Cancri e, nan
5	Age	Idade do passageiro (em anos)	float64	Numérica	0.0 – 79.0
6	VIP	Indica se o passageiro pagou por privilégios VIP (True = sim, False = não)	object	Categórica	False, True, nan
7	RoomService	Valor gasto pelo passageiro com serviço de quarto (em créditos espaciais)	float64	Numérica	0.0 – 14327.0
8	FoodCourt	Valor gasto na praça de alimentação da nave (em créditos espaciais)	float64	Numérica	0.0 – 29813.0
9	ShoppingMall	Valor gasto em lojas a bordo (em créditos espaciais)	float64	Numérica	0.0 – 23492.0
10	Spa	Valor gasto com serviços de spa (em créditos espaciais)	float64	Numérica	0.0 – 22408.0
11	VRDeck	Valor gasto com entretenimento de realidade virtual (em créditos espaciais)	float64	Numérica	0.0 – 24133.0
12	Name	Nome completo do passageiro	object	Categórica	Maham Ofracculy, Juanna Vines, Altark Susent, Solam Susent, Willy Santantines, Sandie Hinetthews, Billex Jacostaffey, Candra Jacostaffey, Andona Beston, Erraiam Flatic
13	Transported	Indica se o passageiro foi transportado para outra dimensão (True = sim, False = não)	bool	Categórica	False, True

Dicionário de dados

SPENDING COLS



Escala logarítmica para spending cols

CÓDIGOS INPUTADES

```
#utilizamos o Homeplanet para realizar um groupby junto com idade para formar uma média e preencher os dados faltantes
df_train['Age'] = df_train.groupby('HomePlanet')['Age'].transform(lambda x: x.fillna(x.mean()))

#preenchimento de dados faltantes
df_train['Age'].fillna(df_train['Age'].mean(), inplace=True)
```

```
# Analisando a relação entre VIP e Destination
vip_destination_crosstab = pd.crosstab(df_train['VIP'], df_train['HomePlanet'])
display(vip_destination_crosstab)
```

HomePlanet	Earth	Europa	Mars
VIP			
False	4680	1958	1653
True	5	131	63

```
vip_destination_crosstab = pd.crosstab(df_train['VIP'], df_train['Destination'])
display(vip_destination_crosstab)
```

Destination	55 Cancri e	PSO J318.5-22	TRAPPIST-1e
VIP			
False	1692	756	5843
True	65	18	116

```
# Imputando valores faltantes em 'VIP' com a moda por 'HomePlanet' e 'Destination'
df_train['VIP'] = df_train.groupby(['HomePlanet', 'Destination'])['VIP'].transform(lambda x: x.fillna(x.mode()[0] if not x.mode().empty else False))
```

```
# Separando a coluna 'Cabin' em 'Cabin_Deck', 'Cabin_Num' e 'Cabin_Side'

df_train[['Cabin_Deck', 'Cabin_Num', 'Cabin_Side']] = df_train['Cabin'].str.split('/', expand=True)

display(df_train[['Cabin', 'Cabin_Deck', 'Cabin_Num', 'Cabin_Side']].head())
```

	Cabin	Cabin_Deck	Cabin_Num	Cabin_Side
0	B/0/P	B	0	P
1	F/0/S	F	0	S
2	A/0/S	A	0	S
3	A/0/S	A	0	S
4	F/1/S	F	1	S

```
# Convertendo 'PassengerGroup' para numérico
df_train['PassengerGroup'] = pd.to_numeric(df_train['PassengerGroup'], errors='coerce')

# Calculando a correlação entre 'Cabin_Num' e 'PassengerGroup'
correlation = df_train['Cabin_Num'].corr(df_train['PassengerGroup'])

print(f"A correlação entre Cabin_Num e PassengerGroup é: {correlation}")
```

A correlação entre Cabin_Num e PassengerGroup é: 0.679723036170993

```
# Preenchendo os valores faltantes na coluna 'Cabin' original
# Apenas para as linhas onde 'Cabin' era NaN, construímos a string usando as colunas imputadas
df_train['Cabin'] = df_train.apply(
    lambda row: f"{row['Cabin_Deck']}/{int(row['Cabin_Num'])}/{row['Cabin_Side']}" if pd.isna(row['Cabin']) else row['Cabin'],
    axis=1
)

# Verificando a quantidade de dados faltantes em 'Cabin' após a atualização
print("Quantidade de dados faltantes em 'Cabin' após atualização:")
display(df_train['Cabin'].isnull().sum())
```

Quantidade de dados faltantes em 'Cabin' após atualização:
np.int64(0)

IMPUTAÇÃO DF_TEST

HomePlanet

```
# Imputar HomePlanet em df_test usando a moda de df_train
mode_homeplanet_train = df_train['HomePlanet'].mode()[0]
df_test['HomePlanet'] = df_test['HomePlanet'].fillna(mode_homeplanet_train)

# Verificar dados faltantes em HomePlanet após imputação
print("Quantidade de dados faltantes em 'HomePlanet' em df_test após imputação:")
display(df_test['HomePlanet'].isnull().sum())
```

Quantidade de dados faltantes em 'HomePlanet' em df_test após imputação:
np.int64(0)

CryoSleep

```
# Imputar CryoSleep em df_test usando a moda de df_train

# Converter para booleano antes de imputar, lidando com NaNs
df_test['CryoSleep'] = df_test['CryoSleep'].astype(bool)

# Calcular a moda no df_train (já feito anteriormente, apenas recuperando o valor)
mode_cryosleep_train = df_train['CryoSleep'].mode()[0]

# Imputar os valores faltantes em df_test
df_test['CryoSleep'] = df_test['CryoSleep'].fillna(mode_cryosleep_train)

# Verificar dados faltantes em CryoSleep após imputação
print("Quantidade de dados faltantes em 'CryoSleep' em df_test após imputação:")
display(df_test['CryoSleep'].isnull().sum())
```

Age

```
# Imputar Age em df_test usando a média agrupada por HomePlanet e a média geral de df_train

# Calculando a média de Age por HomePlanet no df_train
mean_age_by_homeplanet_train = df_train.groupby('HomePlanet')['Age'].mean()

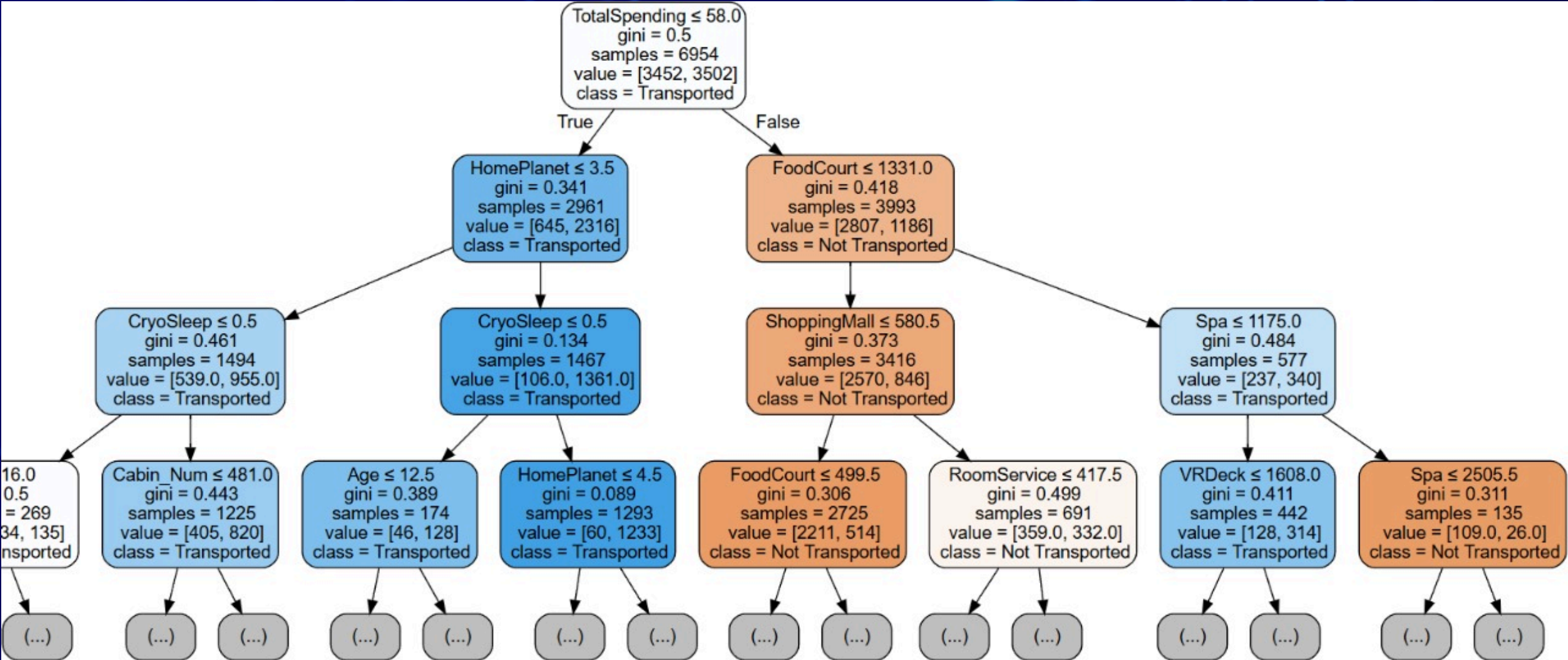
# Imputando valores faltantes em 'Age' no df_test com a média por 'HomePlanet' calculada no df_train
df_test['Age'] = df_test.groupby('HomePlanet')['Age'].transform(lambda x: x.fillna(mean_age_by_homeplanet_train.get(x.name, df_train['Age'].mean())))

# Preencher quaisquer NaNs remanescentes com a média geral de Age do df_train
df_test['Age'].fillna(df_train['Age'].mean(), inplace=True)

# Convertendo a coluna 'Age' para inteiro após a imputação
df_test['Age'] = df_test['Age'].astype(int)

# Verificar dados faltantes em Age após imputação
print("Quantidade de dados faltantes em 'Age' em df_test após imputação:")
display(df_test['Age'].isnull().sum())
```

MODELO: ÁRVORE DE DECISÃO



RESULTADO ÁRVORE

CÓDIGOS DA ÁRVORE

```
# Calculando 'TotalSpending', a que coluna que adicionamos, para df_test
df_test['TotalSpending'] = df_test[spending_cols].sum(axis=1)

# Treino / teste (80-20)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) # stratify=y po

# Declarando o modelo
model_arvore = DecisionTreeClassifier(random_state=42)

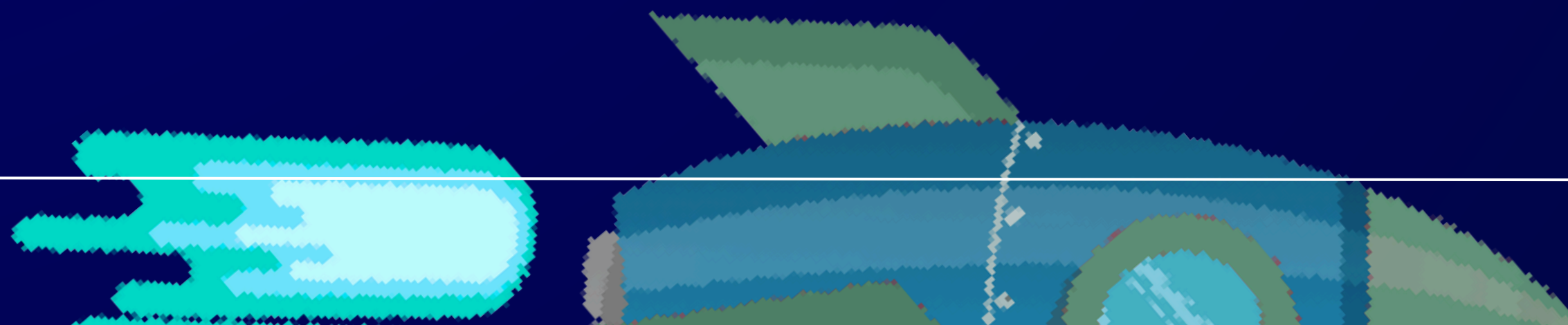
# Treinando o modelo com os dados de TREINO
model_arvore.fit(X_train, y_train)

# Verificando a eficácia do modelo
y_pred_val = model_arvore.predict(X_val)
acc_val = accuracy_score(y_val, y_pred_val)
print(f"Acurácia: {acc_val:.4f}")
print("\n Relatório:")
print(classification_report(y_val, y_pred_val))
```

```
# Predição no df_test
df_test_predicao = df_test[features].astype(X.dtypes) # Para assegurar que es

df_test['Transported_Predicted'] = model_arvore.predict(df_test_predicao)
df_test[['PassengerId', 'Transported_Predicted']].head()
```

	PassengerId	Transported_Predicted
0	0013_01	True
1	0018_01	False
2	0019_01	True
3	0021_01	True
4	0023_01	True



```
# Pré-processamento
categorical_features = ['HomePlanet', 'CryoSleep', 'Cabin_Deck', 'Cabin_Side', 'Destination', 'VIP']
numerical_features = ['Cabin_Num', 'Age', 'RoomService', 'FoodCourt', 'ShoppingMall', 'Spa', 'VRDeck']

preprocessor = ColumnTransformer(
    transformers=[
        ('onehot', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('passthrough', 'passthrough', numerical_features)
    ]
)

# pipeline
dt_model = DecisionTreeClassifier(random_state=42)

pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', dt_model)
])

# Treino / Teste (80-20)
X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Tuned DecisionTree acc=0.7781 (+/- 0.0074)

Top-20 importâncias:

Deck_D	0.445446
CryoSleep_False	0.095493
CryoSleep_True	0.066780
HasCabinNum_1	0.060093
Deck_A	0.035771
IsAlone_0	0.032654
Deck_B	0.030902
HomePlanet_Mars	0.029104
Deck_G	0.027450
Deck_C	0.026211
HomePlanet_Europa	0.017150
HomePlanet_Earth	0.016395
RoomService	0.015355
Deck_T	0.014180
Spa	0.012922
Age	0.012845
Deck_E	0.010195
Deck_F	0.009502
Side_S	0.007628
VIP_True	0.004491

```
# GridSearchCV para tunar a árvore
param_grid = {
    'classifier__criterion': ['gini', 'entropy', 'log_loss'],
    'classifier__max_depth': [4, 6, 8, 10, 12, None],
    'classifier__min_samples_split': [2, 5, 10, 20],
    'classifier__min_samples_leaf': [1, 2, 5, 10],
    'classifier__class_weight': [None, 'balanced']
}

grid_search = GridSearchCV(
    pipeline,
    param_grid,
    cv=5,                # 5-fold cross-validation
    n_jobs=-1,           # usa todos os núcleos disponíveis
    scoring='accuracy',
    verbose=1
)

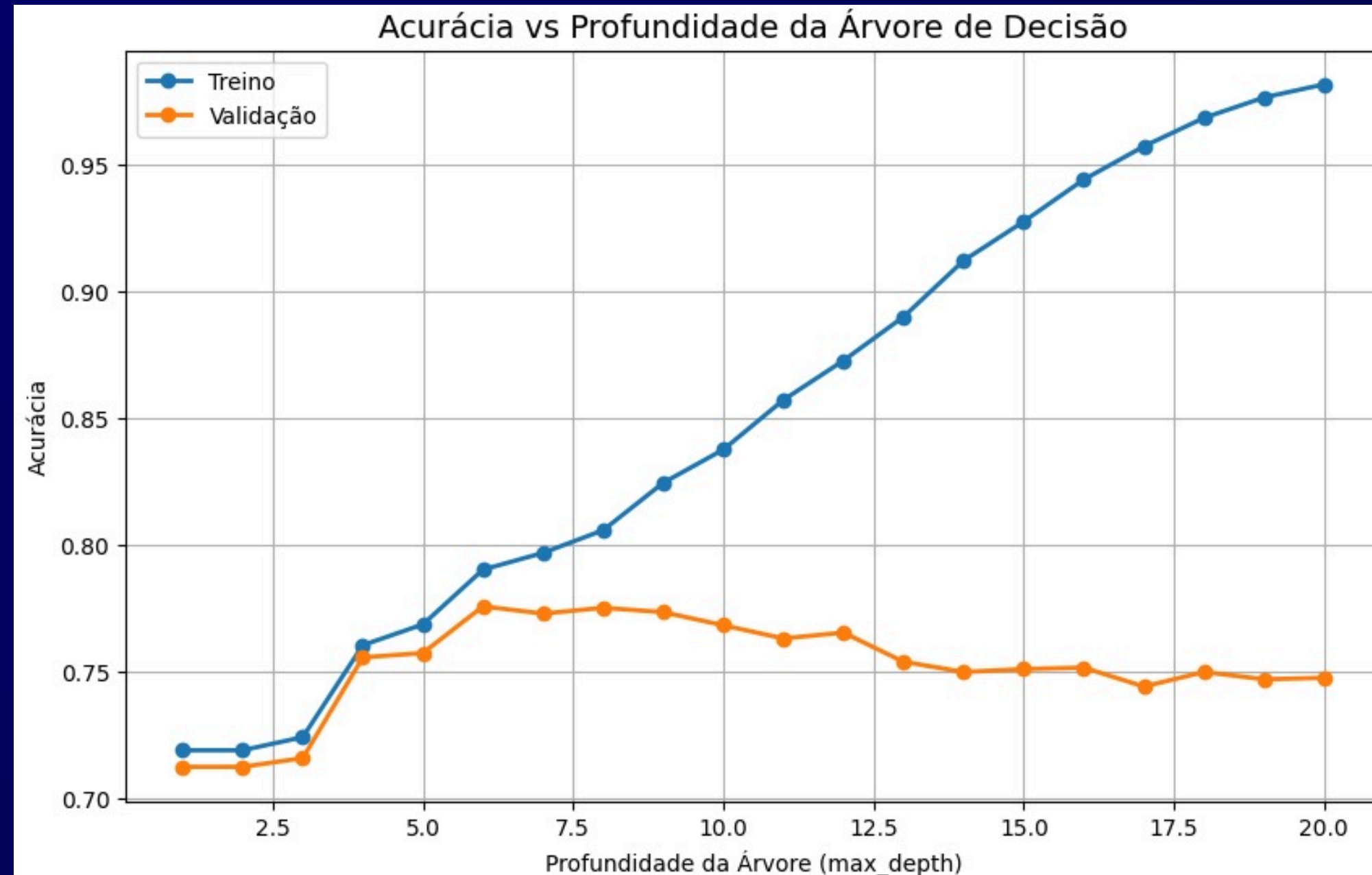
grid_search.fit(X_train, y_train)

print(f"\n Melhor combinação de parâmetros:\n{grid_search.best_params_}")
print(f"Melhor acurácia média de validação: {grid_search.best_score_:.4f}")
```

Pré-processamento, pipeline, boosts

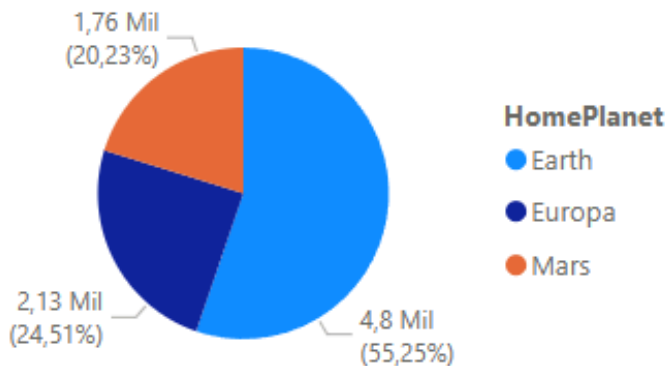
MODELOS AVANÇADOS

AUMENTO DA ACURÁCIA X PROFUNDIDADE

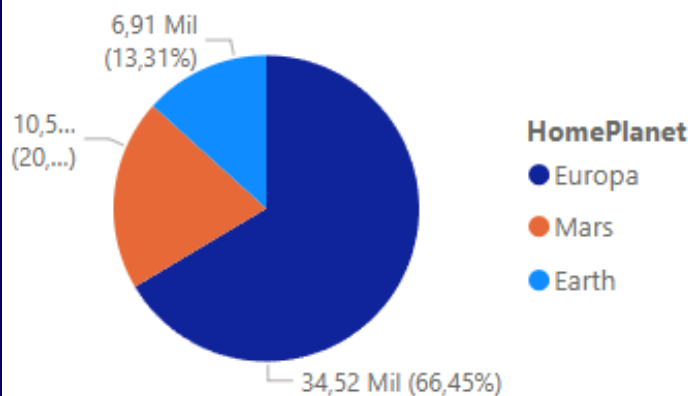


Modelo anterior

Contagem de HomePlanet por HomePlanet



Média de TotalSpending por HomePlanet



Earth

Contagem de Transported
4,803 Mil

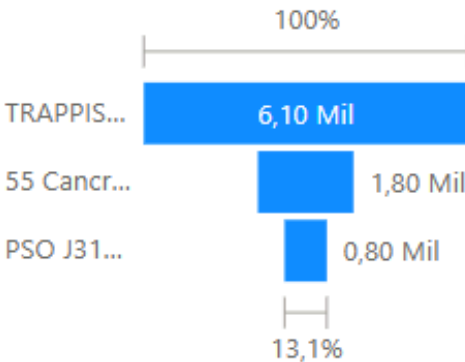
Europa

Contagem de Transported
2,131 Mil

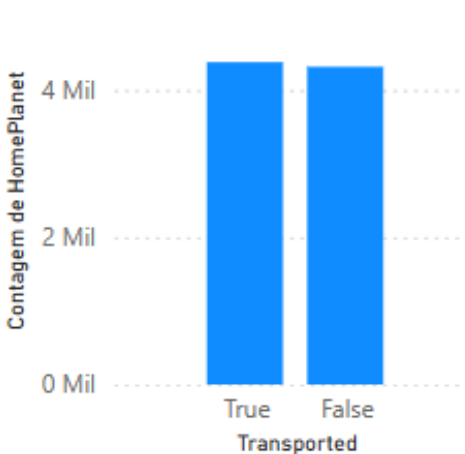
Mars

Contagem de Transported
1,759 Mil

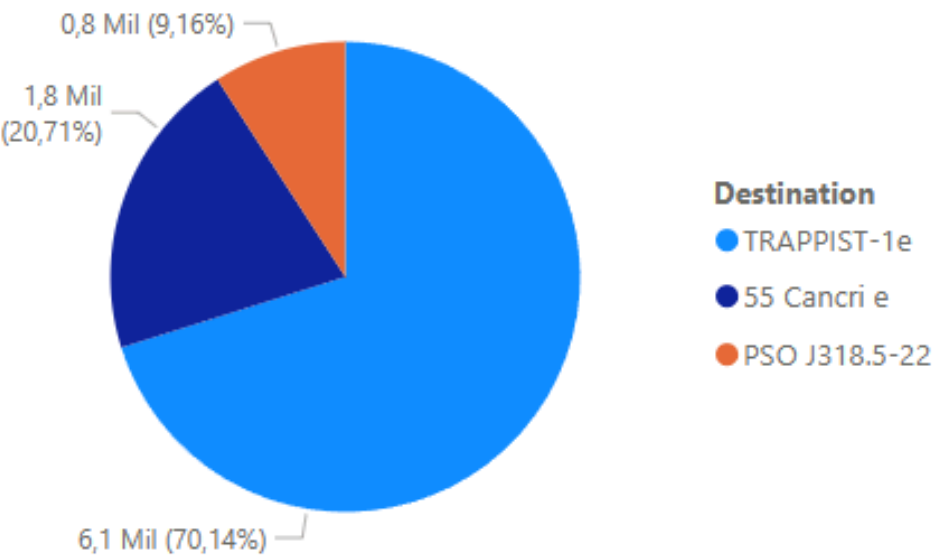
Contagem de Transported por Destination



Contagem de HomePlanet por Transported

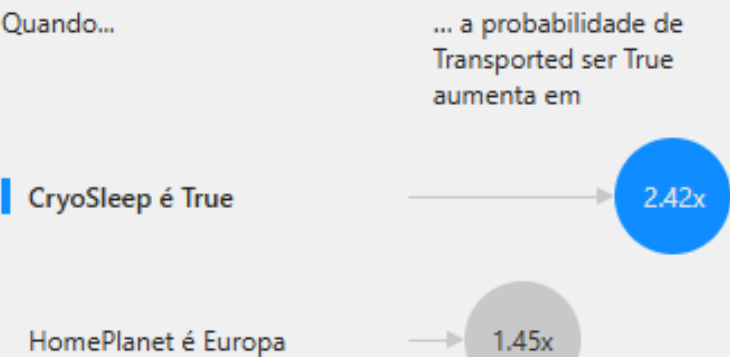


Contagem de Transported por Destination

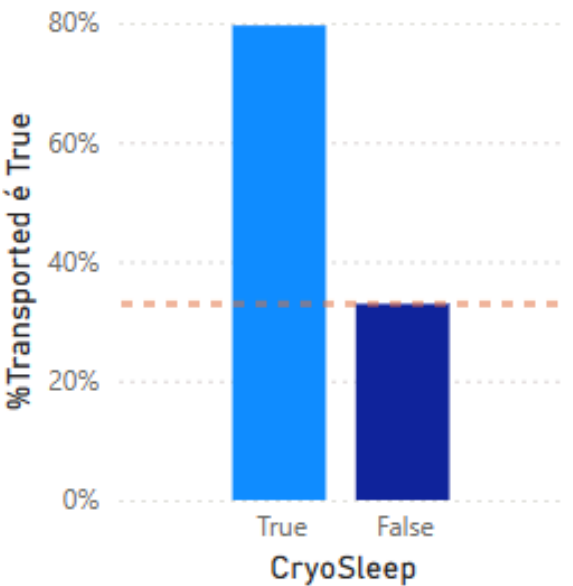


Principais influenciadores Principais segmentos

O que influencia Transported a ser True

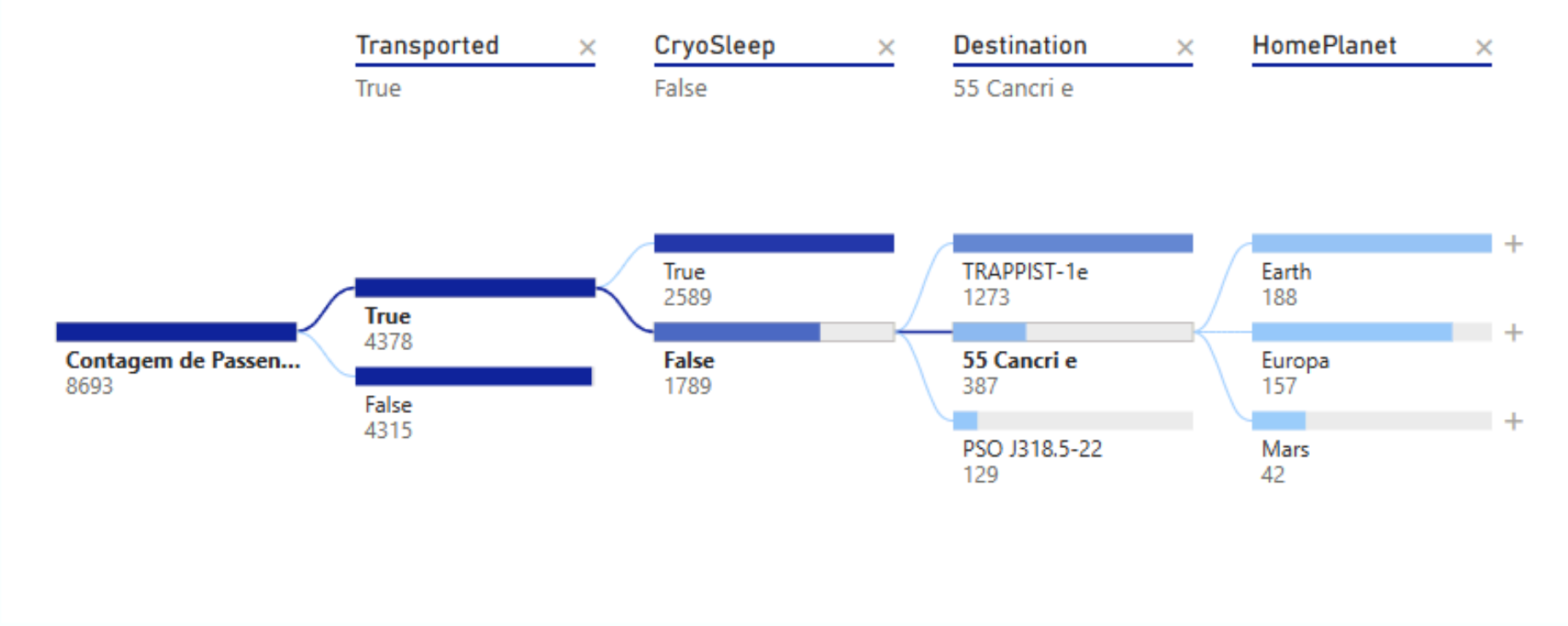


← É mais provável que Transported seja True quando CryoSleep é True do que o contrário (normalmente).



☐ Mostrar apenas os valores que são influenci...

Visualização na forma de árvore



TRABALHO FINAL

ANÁLISE DE DADOS

ENZO SENATORI - 10427175
RAFAEL DE OLIVEIRA CORRÊA - 10438174

ΘBRIGADEΘ
